

Electrum: UC Fail-Stop Server-Supported Signatures

Nikita Snetkov^{1,2}[0000–0002–1414–2080], Jelizaveta Vakarjuk^{1,2}[0000–0001–6398–3663], and Peeter Laud¹[0000–0002–9030–8142]
{nikita.snetkov,jelizaveta.vakarjuk,peeter.laud}@cyber.ee

¹ Cybernetica AS, Mäealuse 2/1, 12618 Tallinn, Estonia

² Tallinn University of Technology, Akadeemia tee 15a, 12618 Tallinn

Abstract. Migration to quantum-safe cryptography represents a significant technological shift, addressing the vulnerabilities of traditional cryptographic primitives, such as KEMs and digital signatures. Yet, a number of challenges remain, especially in the development of secure solutions for sophisticated cryptographic applications. One of them is Smart-ID, European server-supported (threshold) signing service. To address this issue, we present **Electrum**, a fail-stop server-supported signature scheme designed to enhance security of existing Smart-ID service. **Electrum** combines multiprime RSA-based signatures with fail-stop features: providing not only unforgeability against classical adversaries but also allowing to prove that a given signature is a forgery made by classical and/or quantum adversaries. Proposed protocol can be seen as a temporary remedy against the quantum threat until standardised threshold signature schemes become a common practice. To prove security of **Electrum**, we introduce a new ideal functionality $\mathcal{F}^{\text{SplFS}}$ for a fail-stop server-supported signing in the Universal Composability model. We show that **Electrum** protocol securely realizes the proposed functionality $\mathcal{F}^{\text{SplFS}}$.

Keywords: Fail-Stop signatures · Server-Supported Signatures · Migration to Quantum-Safe Cryptography · Quantum Threat · RSA · Threshold Signatures · Universal Composability

1 Introduction

In August 2024, National Institute of Standards and Technology (NIST) has published a set of new standards: a key-encapsulation mechanism – ML-KEM [34]; and two signature schemes – ML-DSA [33] and SLH-DSA [35]. These algorithms are considered to be secure against both classical and quantum adversaries. Finalised release of these standards indicates a new stage of the full-scale migration to quantum-safe cryptography. Multiple companies are already embedding post-quantum algorithms in their products: Google Chrome supports ML-KEM by default ³; Amazon Web Services implemented ML-KEM validation in their AWC-

³ <https://security.googleblog.com/2024/08/post-quantum-cryptography-standards.html>

LC cryptographic library ⁴; Cloudflare already employs post-quantum cryptographic (PQC) protocols for the client-server communications ⁵.

Even though current PQC migration process is not the first cryptographic migration in the history [7], it definitely could compete for the title of “*the most challenging one*”. Compared to the quantum-vulnerable schemes like RSA, ElGamal and ECDSA, the sizes of keys, ciphertexts, and signatures are notably larger for newly proposed schemes [37,2]. Another concern is whether and how systems should support both quantum-safe and quantum-vulnerable protocols simultaneously (i.e. work in *hybrid mode*) ⁶ [30]. Moreover, new standards cannot substitute cryptographic protocols for distinctive cases as i-voting, multiparty computation and distributed signing [15,48].

One of the real-life applications of distributed signing, Smart-ID ⁷ could be seen as a such distinctive case. Smart-ID provides an authentication and a digital signing service to millions of users in Belgium, Iceland and Baltic states. Moreover, Smart-ID is a Qualified Signature Creation Device (QSCD), i.e. signatures created with Smart-ID are legally equivalent to the handwritten signatures in the European Union. Smart-ID is built upon Buldas et al.’s [14] server-supported RSA signing, which is a variant of 2-out-of-2 RSA signature scheme. To substitute RSA in Smart-ID with a quantum-safe scheme, the resulting protocol **must** produce signatures verifiable by a standardised software (i.e. be functionally interchangeable) to enable obtaining QSCD status for the new protocol.

Post-quantum threshold signatures and multisignatures Threshold signature protocols [21] allow any subset of t participants among n key holders to produce a signature but any subset $t - 1$ parties cannot sign a message. Multisignatures [4] are considered a special case when all n signers must participate in a protocol to create a signature (i.e. $t = n$). Lattice-based threshold signatures and multisignatures have garnered significant attention in the recent years: there are several distributed protocols based on variants of ML-DSA [19,20,13,26,47,1,45,43,31,44,42] or other signature schemes [6,18,10,25,24,23,17]. Only three approaches, Trilithium [22], Bienstock et al. [8] and Borin et al. [11] allow to produce standard ML-DSA signatures. These protocols may be potential substitutes of server-supported RSA in Smart-ID. However, required bandwidth, support of hardware security modules and mobile devices cause a stall in immediate deployment (similarly to PQ signatures in TLS 1.3 ⁸).

Fail-stop signatures There is an alternative approach that could *partly mitigate* attacks from both classical and quantum adversaries, which does not require a drastic change in existing infrastructure. Fail-stop signature (FSS) is a type of digital signature scheme with additional functionality: it allows signer to prove

⁴ <https://aws-news.com/article/0193b168-c9fb-aa94-7836-3194fbe864de>

⁵ <https://blog.cloudflare.com/post-quantum-to-origins/>

⁶ https://groups.google.com/a/list.nist.gov/g/pqc-forum/c/6_D0mMSYJZY/m/3DwwIAJXAwAJ

⁷ <https://www.smart-id.com/>

⁸ <https://www.chromium.org/Home/chromium-security/post-quantum-pki-design/>

that an adversary with unlimited resources successfully forged a signature. In context of transition to quantum-safe cryptography, FSS could be used as a temporary solution to guarantee some protection to the signers (i.e. proving that someone managed to produce a signature under their name and revoking detected signatures), until suitable post-quantum alternatives are deployed and certified.

There exist several works on the topic of FSS [40,38,39,3,36], Boschini et al. even considered quantum adversaries for their protocol [12]. The main downside of aforementioned schemes – they are mainly built upon *one-time* primitives. Recently, Yaksetig proposed an ECDSA fail-stop signature scheme [49] with incremental changes to standard ECDSA scheme. The main idea behind their approach is a generation of the secret key from the seed which is then embedded in every signature via padding and one-way function (OWF). Yaksetig’s protocol could potentially be a interim solution for Smart-ID but it lacks: (a) formal security proof (b) security against malicious signer i.e. any signer is able to create signatures that later could be proven to be forgeries (c) support for RSA signatures. Given all the explanations above, we are aiming to ask the following question in this paper:

“Can we construct RSA fail-stop server-supported signing scheme such that: 1. it is compatible with the existing Smart-ID solution, 2. it gives security guarantees against external classical and quantum adversaries, and 3. it provides security against classical malicious client and server?”

Our contribution In this work, we are giving a positive answer to this question. First, we define a new ideal functionality $\mathcal{F}^{\text{SplFS}}$ for the fail-stop server-supported signing. We show that the ideal functionality achieves desired security properties:

- **Security for the signer** – guarantees that if an attacker can generate a valid forgery, then a signer can generate a *proof of forgery* showing that the hardness assumption underlying the scheme’s security has been broken.
- **Security for the recipient** – prevents the possibility of a malicious signer (client) being able to sign a message and subsequently revoke signature by producing a proof of forgery.

Secondly, we present **Electrum**⁹, a RSA-based fail-stop server-supported signing protocol. The signatures generated by this protocol are functionally interchangeable with the standard RSA signatures. We show that **Electrum** securely realizes the $\mathcal{F}^{\text{SplFS}}$ in the universal composability model.

2 Preliminaries

Notation $x \leftarrow \$ X$ denotes sampling an element x uniformly at random from the set X . The symbol $\parallel$ represents a parallel execution of Turing machines or protocols. $\mathcal{Z}$ denotes the environment; $\mathcal{S}$ and $\mathcal{A}$ the ideal and real adversaries.

⁹ The name of the protocol is inspired by Brandon Sanderson’s *Mistborn* series, where electrum, an alloy of gold and silver, was used to partly counter the "time-prediction" abilities.

A *pseudorandom permutation* (PRP) over a domain $\mathcal{D}$ is a function $F : \mathcal{D} \times \{0, 1\}^\kappa \rightarrow \mathcal{D}$, such that $F(\cdot, k)$ is a permutation for each k , and for $k \leftarrow \{0, 1\}^\kappa$ it is indistinguishable from a random permutation over $\mathcal{D}$. As in Bellare et al. [5] and Black et al. [9], pseudorandom permutations over various $\mathcal{D}$ can be constructed from block ciphers.

A *digital signature scheme* is a cryptographic scheme consisting of three algorithms: key generation algorithm KGen that on the security parameter as input λ , outputs a new keypair (pk, sk) ; signing algorithm Sign that on inputs (sk, M) returns a signature σ on the message M ; and the verification algorithm Ver that on inputs (pk, M, σ) either accepts or rejects given signature.

2.1 Server-Supported RSA

Buldas et al.'s [14] server-supported signing protocol is based on multiprime RSA [27]. It is run between two parties, client and server, and could be seen as a special case of two-party signing with additional security properties – *client key-splitting*, *security against offline dictionary attacks* and *clone detection*.

In [14], the public key is of the form $(n_1 \cdot n_2, e)$, where n_1 and n_2 are RSA moduli, generated by the client and the server, respectively. Server knows its private exponent $d_2 = e^{-1} \pmod{\phi(n_2)}$, while client's private exponent $d_1 = e^{-1} \pmod{\phi(n_1)}$ is additively shared as $d'_1 + d''_1 \equiv d_1 \pmod{\phi(n_1)}$ between the client and the server. Client does not store d'_1 , but a random value u , such that $d'_1 = F_{n_1}(u, \text{PIN})$ for a user-provided PIN and a pseudorandom permutation F_{n_1} over the domain $\mathbb{Z}_{n_1}$ (not distinguished from $\mathbb{Z}_{n_1}^*$). The sharing of d_1 protects against offline attacks, because an attacker that has somehow obtained u cannot recognize the correct guess of PIN. Attacker would have to contact the server to verify the guess.

To sign a message msg , the client uses RSA Probabilistic Signature Scheme (PSS) [29] to pad it as m , computes $s'_1 \leftarrow m^{d'_1} \pmod{n_1}$ and sends (msg, s'_1) to server. Server also pads msg to m , computes $s_1 \leftarrow s'_1 \cdot m^{d''_1} \pmod{n_1}$ and verifies that $s_1^e = m \pmod{n_1}$. If verification fails, then server assumes that PIN was incorrectly entered. If verification succeeds then the server computes $s_2 \leftarrow m^{d_2} \pmod{n_2}$, and combines s_1 and s_2 to the final signature using the Chinese Remainder Theorem (CRT).

In order for server to detect clones of the client, the latter sends the former a random value w agreed upon the previous signing session; this value is updated at the current signing session. As the client executes on a not-too-secure platform (a smartphone), its *memory may leak*, while the adversary does not yet obtain full control over the client. If *encrypted memory* leaks, then the adversary learns u and w . If *unencrypted memory* leaks, then the adversary also learns d'_1 .

2.2 Universal composability

Our proofs are given in the *universal composability* (UC) framework [16], though our approach could be seen equivalent to *reactive simulatability* [41]. Using this framework we show that the real system realizing the protocols is *at least as*

Parameter: Description of algorithm $\mathfrak{X}$ (possibly superpolynomial)
Switching on: On input $(\text{switch-on}, \text{sid})$ over the respective input port (connecting to a machine in Sys or the ideal functionality $\mathcal{F}$): record (ON).
Computing: On input $(\text{compute}, \text{sid}, v)$ from $\mathcal{A}$: If (ON) is not recorded, then ignore the input. Otherwise set $w \leftarrow \mathfrak{X}(v)$ and output $(\text{result}, \text{sid}, w)$.

Fig. 1. Helper functionality $\mathcal{F}^{\text{helper}}$

secure as the ideal system. Both systems consist of probabilistic (polynomial-time) interactive Turing machines (PITMs) with connections between them. For every machine, there are interfaces consisting of named input and output ports (the names of which one typically elides) offered to the (*outside*) *environment* and the *adversary*, both of which are PITMs as well or collections of them. A system Sys is considered to be *at least as secure as* the system $\mathcal{F}$, if for each (“*real*”) adversary $\mathcal{A}$, there exists an (“*ideal*”) adversary $\mathcal{S}$, such that for each environment $\mathcal{Z}$, its views in the executions of the collections $\mathcal{Z} \parallel Sys \parallel \mathcal{A}$ and $\mathcal{Z} \parallel \mathcal{F} \parallel \mathcal{S}$ are indistinguishable. In security proofs, the existence of $\mathcal{S}$ is shown by defining a single PITM Sim (the “*simulator*”), and positing $\mathcal{S} := Sim \parallel \mathcal{A}$. Then it is shown that $\mathcal{Z} \parallel \mathcal{A}$ cannot distinguish between Sys and $\mathcal{F} \parallel Sim$.

Our UC security proofs for fail-stop server-supported signatures follow the approach of Nomura and Nakamura [36] for modelling adversaries with certain superpolynomial capabilities. We introduce a helper functionality $\mathcal{F}^{\text{helper}}$ that responds to the queries made by the adversary $\mathcal{A}$ (or $\mathcal{S}$). The algorithm used to respond to the adversary’s queries is part of $\mathcal{F}^{\text{helper}}$: it does not have to be polynomial-time. In the context of quantum adversaries and RSA, it is natural to consider $\mathcal{F}^{\text{helper}}$ that receives a natural number n from the adversary, and responds with $\mathfrak{X}(n)$ — the list of prime factors of n .

We want to be able to discuss provided security “*before*” and “*after*” of the superpolynomial functionality becoming available. We hence state that besides responding to adversary’s queries, $\mathcal{F}^{\text{helper}}$ has an input port, over which it takes the command that “*switches it on*”. The corresponding output port is part of the real system Sys or the ideal functionality $\mathcal{F}$. See Fig. 1 for full description.

As $\mathcal{F}^{\text{helper}}$ is not part of the real/ideal system, the definition of “*at least as secure as*” changes, and becomes relativized with respect to $\mathcal{F}^{\text{helper}}$. We now require that for each $\mathcal{A}$, there exists $\mathcal{S}$, such that no $\mathcal{Z}$ can distinguish the executions of $\mathcal{Z} \parallel Sys \parallel \mathcal{A} \parallel \mathcal{F}^{\text{helper}}$ and $\mathcal{Z} \parallel \mathcal{F} \parallel \mathcal{S} \parallel \mathcal{F}^{\text{helper}}$. Note that if $\mathcal{S}$ is defined as $Sim \parallel \mathcal{A}$, then in the collections with $\mathcal{F}$, it will still be $\mathcal{A}$ that queries $\mathcal{F}^{\text{helper}}$.

3 Ideal Functionality for Fail-Stop Server-Supported Signatures

In this section, we present ideal functionality $\mathcal{F}^{\text{SpIFS}}$ for a fail-stop server-supported signing. Since we aim to introduce additional fail-stop features on top of exist-

ing distributed signing protocol, we base $\mathcal{F}^{\text{SplFS}}$ on Snetkov et al. [46] server-supported signing ideal functionality $\mathcal{F}^{\text{gSpl}}$. Their functionality $\mathcal{F}^{\text{gSpl}}$ models not only the security of two-party threshold signatures, but different corruption levels for one of the parties (*a client*), a clone detection and PIN verification. Fail-stop server-supported signatures have even more complex properties, which are reflected in the design of the $\mathcal{F}^{\text{SplFS}}$. Similarly to $\mathcal{F}^{\text{gSpl}}$, our functionality $\mathcal{F}^{\text{SplFS}}$ is defined for n parties (in the environment $\mathcal{Z}$), all of which are able to verify produced signatures. Also similarly, parties P_1 and P_2 have the roles of the client and the server. As we see in the following sections, the fail-stop scenario involves more parties (P_3 and P_4) with a special roles (revocation service and alerter).

The initialization and key generation operations in $\mathcal{F}^{\text{SplFS}}$ are defined in Fig. 2. The initialization shows the internal state of $\mathcal{F}^{\text{SplFS}}$: there is the corruption flag $c_S \in \{0, 1\}$ for the server, and the corruption gauge $c_P \in \{0, 1, 2, 3\}$ for the client. Besides the adversary having taken over the client (encoded by $c_P = 3$), we want to model weaker corruptions/attacks, because the client program is expected to run on a vulnerable platform (a smartphone). Identically to [14,46], we consider the leaks of encrypted or unencrypted memory of the client ($c_P = 1$ or $c_P = 2$).

The first kind of leak is meaningful, because we assume the memory to be encrypted with a PIN picked from the set $\{0, \dots, L - 1\}$, i.e. the encryption keys have a very small entropy. The count of wrong PIN entries is stored in T , with T_0 being the limit. In the case of the second kind of leak, if client's unencrypted memory has leaked, then the adversary is able to pretend as a client. As long as the adversary does not fully control the client, the *clone detection* mechanism employed by the server can be successful. To model this, the bit b_{lq} records whether the last signing query was initiated by the environment or by the adversary.

The fail-stop properties of $\mathcal{F}^{\text{SplFS}}$ are expected to protect against the appearance of a mechanism that can solve a particular hard problem, e.g. factoring. The bit b_{sp} indicates whether it is still safe to base cryptography on the hardness of that problem. However, we expect there to be other hard problems (e.g. distinguish AES from a random permutation) that will never become easy.

We see that in key generation, the algorithm `Ver` is received from the adversary. It plays the role of the public key; this formulation is standard for UC signatures. Also at the key generation, the client specifies the `PIN` that later gives access to the signing functionality.

The adversary may corrupt either the client or the server. These corruptions may happen anytime, hence the adversary may be considered *adaptive*. In case of corrupting the client, the clone detection mechanism will be reset; the next signing query may come from either the client or the adversary. The order of corruptions may vary, and several different actions by the adversary may raise the corruption level of one of the parties. At certain levels, certain actions by $\mathcal{F}^{\text{SplFS}}$ must happen, and Fig. 2 specifies these in the form of *triggers*. These triggers will be set off the first time the condition becomes true. In particular, if the first one is set off, then the second one also triggered. These triggers are easy to justify:

Initialization: On input (init) from P_3 , set $c_S := c_P := T := 0$; $b_{lq} := b_{OK} := b_{sp} := 1$; $Ver := \perp$; $T_{sign} := \{\}$. Send (init) to $\mathcal{S}$.

Key Generation: On input (keygen, sid, L, T_0 , PIN) from client and (keygen, sid, T_0) from server, if Ver is already recorded or $PIN \notin \{0, \dots, L-1\}$, ignore this query. Otherwise:

- Send (keygen-init) to adversary $\mathcal{S}$.
- Upon receiving (key, Ver) from $\mathcal{S}$, where Ver is signature verification algorithm, store (Ver, L, PIN, T_0) and send (key, Ver) to both client and server.

Corrupt server: On input (corrupt-server) from $\mathcal{S}$: set $c_S := 1$ and send (corrupt-server) to server.

Corrupt client: On input (corrupt-client, ℓ) from $\mathcal{S}$, where $\ell \in \{1, 2, 3\}$:

- If $c_P > \ell$, ignore this query.
- Set $b_{lq} := \perp$. If $Ver = \perp$, set $c_P := 3$. Otherwise, set $c_P := \ell$. Send (corrupt-client, ℓ) to client.

Derived values: $b_{TC} \equiv b_{sp} \wedge ([c_S = 0] \vee [c_P = 0])$. $b_{KG} \equiv [Ver \neq \perp]$.

Trigger: If $c_S = c_P = 1$ then set $c_P := 2$.

Trigger: If $c_P \geq 2$ and b_{KG} then send (corrupt-pin, PIN) to $\mathcal{S}$.

Forwarding: If $c_P = 3$ [resp. $c_S = 1$], then inputs from client [resp. server] are forwarded to $\mathcal{S}$; and $\mathcal{S}$ determines the outputs that client [resp. server] receives.

Fig. 2. Ideal functionality for fail-stop server-supported signing $\mathcal{F}^{SpIFS}$ (1/4)

if the server is corrupted (i.e. the adversary has access to server's keyshare) and the adversary also has client's encrypted keyshare, then the adversary may try out all PINs in order to find client's unencrypted keyshare. Adversary could do it since the correctness of the unencrypted keyshare may be checked against the public key. Also, if client's unencrypted keyshare has leaked to the adversary, then it is natural to assume that the encrypted keyshare has leaked as well, thus allowing the adversary to brute-force the PIN.

The signing-related operations of $\mathcal{F}^{SpIFS}$ are quite similar to [46], they are given in Fig. 3. They only work if key generation has taken place ($b_{KG} = 1$). We see that only the client and the server can generate signatures, while the adversary can substitute for one of them. An honest server will always perform a clone detection check, while any client must provide the correct PIN to $\mathcal{F}^{SpIFS}$ (even if the server is dishonest) to create a signature. The signatures can be created also when both the client and the server have been corrupted, but in this case, $\mathcal{F}^{SpIFS}$ does not need to be contacted at all. Such signatures are processed at the verification.

We gather generic subroutines in Fig. 4. The **process-pin** is for the checking of PIN that manages the counter of false attempts and send back failure notices. Subroutine **clone-check** compares the initiator of the current signing attempt against the previous one (note that $b_{lq} \in \{\perp, 0, 1\}$). There is also the subroutine

Signing by client and server: On input $(\text{sign}, \text{sid}, M, \text{PIN}')$ from the client and $(\text{sign}, \text{sid})$ from the server, ignore unless $b_{\text{KG}} \wedge b_{\text{OK}}$. Otherwise:

- Call `clone-check(1)` and `process-pin(PIN')`. If successful, compute σ using `request-sig(sid, M)`.
- If $\sigma \neq \perp$, output $(\text{sign-success}, \text{sid}, M, \sigma)$ to client, and $(\text{signature}, \text{sid}, M, \sigma)$ to server. Otherwise, return $(\text{sign-fail}, \text{sid})$ to client and server.

Signing by client and adversary: On input $(\text{sign}, \text{sid}, M, \text{PIN}')$ from the client and $(\text{sign-server}, \text{sid})$ from $\mathcal{S}$, ignore unless $b_{\text{KG}} \wedge [\text{cs} = 1]$. Otherwise:

- Call `process-pin(PIN')` and then compute σ by querying `request-sig(sid, M)`.
- If $\sigma \neq \perp$, output $(\text{sign-success}, \text{sid}, M, \sigma)$ to client. Otherwise, return $(\text{sign-fail}, \text{sid})$ to both client and $\mathcal{S}$.

Signing by adversary and server: On input $(\text{sign-client}, \text{sid}, M, \text{PIN}')$ from $\mathcal{S}$ and $(\text{sign}, \text{sid})$ from the server: ignore unless $b_{\text{KG}} \wedge b_{\text{OK}} \wedge [\text{cp} > 0]$. Otherwise go to step $\text{cp} \in \{1, 2, 3\}$:

1. Call `clone-check(0)` and `process-pin(PIN')`. If successful, set $\text{cp} := 2$ and go to step 3.
2. Call `clone-check(0)`. If successful, go to step 3.
3. Compute σ using `request-sig(sid, M)`. If $\sigma \neq \perp$, output $(\text{signature}, \text{sid}, M, \sigma)$ to server. Otherwise, return (sign-fail) to both $\mathcal{S}$ and server.

Verification: On input $(\text{verify}, \text{sid}, M, \sigma)$ from any party P , define the value b_{out} as follows:

1. If $\exists c$, s.t. $\text{T}_{\text{sign}}(M, \sigma) = c$, then set $b_{\text{out}} := [c \neq 0]$
2. Otherwise, if b_{TC} , and $\neg \exists \sigma', c' \geq 1$, s.t. $\text{T}_{\text{sign}}(M, \sigma') = c'$, then set $b_{\text{out}} := c := 0$
3. Otherwise, set $b_{\text{out}} := \text{Ver}(M, \sigma)$. If $\neg b_{\text{sp}}$ and $[\text{cs} = 1] \vee [\text{cp} \geq 2]$, then set $c := b_{\text{out}}$, otherwise set $c := b_{\text{out}} \cdot (2 + b_{\text{sp}})$.

Update $\text{T}_{\text{sign}}(M, \sigma) := c$ and output $(\text{is-verified}, \text{sid}, M, \sigma, b_{\text{out}})$ to party P .

Fig. 3. Ideal functionality for fail-stop server-supported signing $\mathcal{F}^{\text{SplFS}}$ (2/4)

`request-sig` for obtaining the actual bit-string σ that serves as the signature. The choice of requesting it from the adversary is the standard one.

The functionality $\mathcal{F}^{\text{SplFS}}$ collects the messages and the signatures in the partial map T_{sign} . For each pair (M, σ) that the functionality has seen due to having produced it, or having received it for verification, the map records that pair's *verification status* $\text{T}_{\text{sign}}(M, \sigma) \in \{0, 1, 2, 3\}$. Here the value 0 means that σ is an invalid signature, while other values indicate a valid signature. Moreover, the value 3 means that the signature will be impossible to revoke later, value 2 means that the signature can be revoked, and value 1 indicates unknown revocability.

Supporting subroutine `process-pin(PIN')`:

- If $\text{PIN} = \text{PIN}'$, set $T := 0$ and give control back to the invoker.
- Otherwise, increment T . If $T \geq T_0$ then set $b_{\text{OK}} := 0$. Return $(\text{sign-fail}, \text{sid})$ to the two parties (out of client, server, and $\mathcal{S}$) that initiated the signing.

Supporting subroutine `clone-check( $d$ )`:

- If $b_{\text{iq}} = 1 - d$, set $b_{\text{OK}} := 0$ and return $(\text{sign-fail}, \text{sid})$ to the two parties that initiated the signing.
- Otherwise, set $b_{\text{iq}} := d$ and return to the invoker.

Supporting subroutine `request-sig( $\text{sid}, M$ )`:

- Send $(\text{sign-init}, \text{sid}, M)$ to $\mathcal{S}$.
- Upon receiving $(\text{signature}, \text{sid}, \sigma)$ from $\mathcal{S}$, check whether $(M, \sigma, 0)$ is already stored. If true, restart `request-sig( $\text{sid}, M$ )`.
- Check if $\text{Ver}(M, \sigma) = 1$. If true, and $b_{\text{TC}} = 1$, set $T_{\text{sign}}(M, \sigma) := 3$ and return σ ; if true, but $b_{\text{TC}} = 0$, return σ ; otherwise return $\perp$.

Fig. 4. Ideal functionality for fail-stop server-supported signing $\mathcal{F}^{\text{SpIFS}}$ (3/4)

The revocation-related operations of $\mathcal{F}^{\text{SpIFS}}$ are presented in Fig. 5, and here we also introduce certain new roles. Our proofs of revocation will not be verifiable publicly, but only by a dedicated *revocation service* that we denote either as RS or P_3 . The outcomes of these verifications should be made available to other parties; this is not handled by $\mathcal{F}^{\text{SpIFS}}$, but should be handled by other components of the larger system that also includes $\mathcal{F}^{\text{SpIFS}}$.

Recall that $\mathcal{F}^{\text{SpIFS}}$ is connected not only with the environment $\mathcal{Z}$ and the adversary $\mathcal{S}$, but also with the helper functionality $\mathcal{F}^{\text{helper}}$ that allows the adversary to solve a certain superpolynomial task. The $\mathcal{F}^{\text{SpIFS}} \rightarrow \mathcal{F}^{\text{helper}}$ connection is used solely to activate the latter, making it to respond the adversary's queries. We have chosen to model the activation by letting the party P_4 (whom we also call *Alerter*) to send a notification to $\mathcal{F}^{\text{SpIFS}}$.

Forgery proofs, similarly to signings, are initiated by the client. The client's inputs to the forgery proving protocol are *two* messages M_1, M_2 and their signature σ_1, σ_2 , but also the PIN, because in the real functionality, it may need to use values in its memory that have been encrypted. The RS learns the messages and signatures, and whether it determined *at least one of them* to be forged. See Sec. 7 for the discussion of its usefulness. The ideal functionality may not always be able to decide whether the revocation protocol is going to succeed or not. Also, the success may also depend on the PINs. In these cases, $\mathcal{S}$ will need to make the decision. But in order to not leak certain values from $\mathcal{F}^{\text{SpIFS}}$ to $\mathcal{S}$, the latter will specify a decision algorithm $\mathfrak{A}$ that will be executed by $\mathcal{F}^{\text{SpIFS}}$.

The functionality $\mathcal{F}^{\text{SpIFS}}$ is designed to enforce certain safety properties of the possible sequences of its inputs and outputs. It can be shown to satisfy

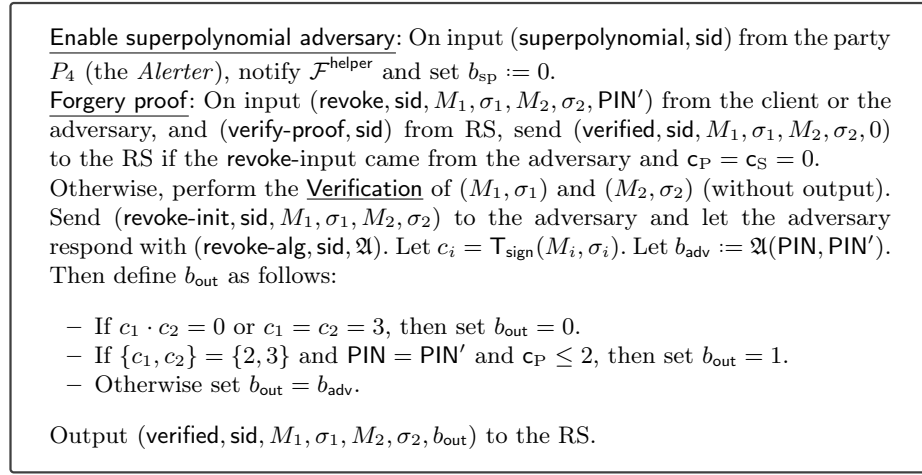


Fig. 5. Ideal functionality for fail-stop server-supported signing $\mathcal{F}^{\text{SplFS}}$ (4/4)

the same unforgeability properties that Snetkov et al. [46, Thm. 1–3] stated for their $\mathcal{F}^{\text{Spl}}$ functionality. Additional fail-stop properties of $\mathcal{F}^{\text{SplFS}}$ are stated in Theorems 1–3.

The first theorem defines non-repudiation property of the Electrum protocol: a non-forged signature (i.e. one created through $\mathcal{F}^{\text{SplFS}}$) cannot later be shown as forged.

Theorem 1. *If an environment $\mathcal{Z}$ and an adversary $\mathcal{S}$ running in parallel with $\mathcal{F}^{\text{SplFS}}$ have not issued the commands to corrupt both the client and the server, then they are not capable of producing signatures σ_1, σ_2 **through signing queries** for some messages M_1, M_2 , such that (verified, sid, $M_1, \sigma_1, M_2, \sigma_2, 1$) would ever be sent to RS by $\mathcal{F}^{\text{SplFS}}$ (for any sid).*

Proof. The message (verified, sid, $M_1, \sigma_1, M_2, \sigma_2, 1$) can be sent to RS only as the result of **Forgery proof** query to $\mathcal{F}^{\text{SplFS}}$. The last component of that message can be “1” only if $T_{\text{sign}}(M_i, \sigma_i) \in \{1, 2\}$. It implies that the entry $T_{\text{sign}}(M_i, \sigma_i)$ must have been defined during a verification query processing, because the values 1 and 2 are assigned to T_{sign} nowhere else in $\mathcal{F}^{\text{SplFS}}$. Hence the signature σ for message M was not produced via a signing query, which is only possible if both client and server are corrupted. $\square$

The second theorem states that forged signatures can be shown as forged, if the client wants to do it (i.e. if $c_P \leq 2$). This property comes with a caveat: if the superpolynomial helper has been switched on, and at least the client or server has been corrupted to the level where the adversary has been capable of reading its memory, then the adversary may have enough information to create unrevocable forgeries.

Theorem 2. *If an environment $\mathcal{Z}$ and an adversary $\mathcal{S}$ running in parallel with $\mathcal{F}^{\text{SplFS}}$ have issued the command to sign some message M_0 resulting in a signature σ_0 , but have not issued the command to sign the message M , and if they also have not issued the commands causing one of the following:*

- (A) *corrupting the client to level $c_P = 3$;*
- (B) *corrupting both the server and the client (to any level of corruption);*
- (C) *enabling superpolynomial adversary, and also corrupting either the server or the client to level ≥ 2 ,*

then there exists no σ , for which the query $(\text{verify}, \text{sid}, M, \sigma)$ would return 1, but the query $(\text{revoke}, \text{sid}, M_0, \sigma_0, M, \sigma, \text{PIN})$ made by the client and the query $(\text{verify-proof}, \text{sid})$ made by the RS would return $(\text{verified}, \text{sid}, M_0, \sigma_0, M, \sigma, 0)$ to the RS, where PIN was used by the client during key generation.

Proof. If a pair (M, σ) has been produced, such that verification outputs 1, but $(\text{verified},$

$\text{sid}, M_0, \sigma_0, M, \sigma, 0)$ is returned to the RS upon query, then $T_{\text{sign}}(M, \sigma) \in \{1, 2, 3\}$ must have been recorded by $\mathcal{F}^{\text{SplFS}}$.

The case when $T_{\text{sign}}(M, \sigma) = 1$ is impossible, since it implies $\neg b_{\text{sp}}$ and $[c_S = 1] \vee [c_P \geq 2]$, which corresponds to an issuance of commands in the set (C).

If $T_{\text{sign}}(M, \sigma) = 2$ and $c_P \leq 2$, then the revoke -command, if issued with the correct PIN , will cause $b_{\text{out}} = 1$ to be returned to the RS. Meaning, signature σ cannot have validity 2.

For the case whether $T_{\text{sign}}(M, \sigma) = 3$ is possible, let define σ_3 to be the chronologically first signature, for which $T_{\text{sign}}(M, \sigma_3) = 3$ holds. The point $T_{\text{sign}}(M, \sigma_3) = 3$ must have been defined while $\mathcal{F}^{\text{SplFS}}$ responds to a verification command. To obtain $c = 3$, the verification execution must have taken the third branch, and $b_{\text{sp}} = 1$. To get to the third branch, the second branch must have been avoided, hence at least one of the following had to be true:

- there already existed some σ' , such that $T_{\text{sign}}(M, \sigma') \geq 1$;
- $b_{\text{TC}} = 0$.

The second case is equivalent to $b_{\text{sp}} = 0$, or both client and server being corrupted. We have excluded this possibility, meaning the first case holds. Due to definition of σ_3 , we have $T_{\text{sign}}(M, \sigma') = 2$. The point $T_{\text{sign}}(M, \sigma') = 2$ similarly must have been defined while $\mathcal{F}^{\text{SplFS}}$ responds to a verification command. But there is no way to obtain $c = 2$ at this time, because superpolynomial adversary was not available ($b_{\text{sp}} = 1$), when verifying σ' . This implies the case $T_{\text{sign}}(M, \sigma) = 3$ is impossible. $\square$

The third theorem states that only signatures “created” (i.e. seen by $\mathcal{F}^{\text{SplFS}}$) after $\mathcal{F}^{\text{helper}}$ has been switched on, can possibly be shown as forgeries.

Theorem 3. *If an adversary $\mathcal{S}$ and an environment $\mathcal{Z}$, running in parallel with $\mathcal{F}^{\text{SplFS}}$, corrupting at most one of the client and the server, submit pairs (M_1, σ_1) and (M_2, σ_2) for verification and receive 1 as the answers before they tell $\mathcal{F}^{\text{SplFS}}$ to turn on $\mathcal{F}^{\text{helper}}$, then $\mathcal{F}^{\text{SplFS}}$ will never send $(\text{verified}, \text{sid}, M_1, \sigma_1, M_2, \sigma_2, 1)$ to RS (for any sid).*

Proof. Let us assume that $\mathcal{F}^{\text{SpIFS}}$ returned $(\text{verified}, \text{sid}, M_1, \sigma_1, M_2, \sigma_2, 1)$ to RS. Similarly to the proof of Thm. 1, we must have $\mathsf{T}_{\text{sign}}(M_i, \sigma_i) \in \{1, 2\}$ defined at some time during the execution.

According to our assumptions, the time of defining $\mathsf{T}_{\text{sign}}(M_i, \sigma_i)$ must have been before $\mathcal{F}^{\text{helper}}$ was turned on, i.e. while $b_{\text{sp}} = 1$. But if $b_{\text{sp}} = 1$, then $\mathcal{F}^{\text{SpIFS}}$ assigns only values 0 or 3 to the points of T_{sign} . $\square$

4 Electrum: Fail-Stop Server-Supported Signature Scheme

We expect a protocol implementing $\mathcal{F}^{\text{SpIFS}}$ to be simple to construct in the $\mathcal{F}^{\text{SpI}}$ -hybrid model, by adding some extra padding to the messages M while signing them. But this would be unsatisfactory from the interoperability perspective: if $\mathcal{F}^{\text{SpI}}$ was securely implemented by the protocol of Buldas et al. [14] (as shown by Snetkov et al. [46]), then we would lose the ability of the signatures being verified by any standards-compliant implementation of RSA.

Instead, Electrum protocol modifies the construction of Buldas et al. [14] by directly modifying the construction of the padding of the messages. Following the ideas of Yaksetig’s fail-stop ECDSA protocol [49], we generate the randomness used in the padding from a seed. The seed for each signature is generated from a master seed. To revoke a signature, the client will reveal to the RS the master seed, and the latter can verify that the padding in the to-be-revoked signature was not created using the master seed, while there exist other signatures created using the master seed. The client is kept honest by the server — as long as the server is honest, it does not participate in the signing of messages with padding from an invalid seed. In this way, the threshold setting allows us to bypass the repudiation issues.

It is surprisingly complicated to store the (encrypted) master seed at the client side in a manner that does not open up the system to offline guessing attacks in case of a leak of the client’s encrypted memory. The randomness used in the padding of a message can be extracted from the signature, and its generation from a seed that was generated from the master seed provides a way to check the guesses of the PIN. To prevent such guesses, a share of the master seed is kept by the server and made available to the client only during a signing session. Moreover, before receiving this share, the client has to convince the server that it knows the PIN.

These ideas are reflected in Electrum’s key generation (Fig. 6), signing (Fig. 7) and revocation (Fig. 8) protocols. These protocols define as a real system (in the sense of Universal Composability) Sys offering an interface to $\mathcal{Z}$ that corresponds to an n -party functionality. The system Sys consists of machines $\mathcal{M}_1, \dots, \mathcal{M}_n$, each of which responds to certain commands from $\mathcal{Z}$. Party P_i in $\mathcal{Z}$ (where $i \in \{1, \dots, n\}$) can verify signatures σ given to messages M by issuing the command $(\text{verify}, \text{sid}, M, \sigma)$ to $\mathcal{M}_i$; that machine runs the RSA-PSS verification algorithm [29, Sec. 8.1.2] on M and σ . Parties P_1 – P_4 are the *client*, the *server*, the *Revocation Service*, and the *Alerter*, respectively.

KeyGen _C (pk _{RS})	KeyGen _S (pk _{RS})
1 : generate primes p_1, q_1	1 : generate primes p_2, q_2
2 : $n_1 \leftarrow p_1 \cdot q_1$	2 : $n_2 \leftarrow p_2 \cdot q_2$
3 : $d_1 \leftarrow e^{-1} \bmod \phi(n_1)$	3 : $C \Rightarrow: \langle d_1'', n_1, \text{rnd}_c \rangle$
4 : $u, u' \leftarrow_{\$} \{0, 1\}^l$	4 : $n \leftarrow n_1 \cdot n_2$
5 : $d_1' \leftarrow F_{n_1}(u, \text{PIN})$	5 : $w \leftarrow_{\$} \{0, 1\}^k$
6 : $d_1'' \leftarrow (d_1 - d_1') \bmod \phi(n_1)$	6 : $\text{rnd}_s \leftarrow_{\$} \{0, 1\}^l$
7 : $\text{rnd}_c \leftarrow F(u', \text{PIN})$	7 : generate randomness r
8 : $\Rightarrow S : \langle d_1'', n_1, \text{rnd}_c \rangle$	8 : $c_{\text{rnd}} \leftarrow \text{Enc}_{\text{pk}_{\text{RS}}}(\text{rnd}_s; r)$
9 : $S \Rightarrow: \langle n, w, c_{\text{rnd}}, \text{rnd}_s, r \rangle$	9 : $\Rightarrow C : \langle n, w, c_{\text{rnd}}, \text{rnd}_s, r \rangle$
10 : check: $n_1 \mid n \wedge c_{\text{rnd}} = \text{Enc}_{\text{pk}_{\text{RS}}}(\text{rnd}_s; r)$	10 : $\text{rnd}_A \leftarrow \text{rnd}_c \oplus \text{rnd}_s$
11 : store $\langle n, n_1, u, u', w, c_{\text{rnd}} \rangle$	11 : store $\langle n, n_1, n_2, d_1'', d_2, \text{rnd}_A, w, \text{rnd}_s, r \rangle$
12 : return pk $\leftarrow \langle n, e \rangle$	12 : return pk $\leftarrow \langle n, e \rangle$

Fig. 6. Electrum Key Generation Protocol

Sign _C (pk _{RS} , msg, PIN)	Sign _S (pk _{RS})
1 : $S \Rightarrow: \langle \text{chal} \rangle$	1 : $\text{chal} \leftarrow_{\$} \mathbb{Z}_{n_1}$
2 : $d_1' \leftarrow F_{n_1}(u, \text{PIN})$	2 : $\Rightarrow C : \langle \text{chal} \rangle$
3 : $\text{resp} \leftarrow \text{Pad}(\text{chal}, w)^{d_1'} \bmod n_1$	3 : $C \Rightarrow: \langle \text{resp}, w' \rangle$
4 : $\Rightarrow S : \langle \text{resp}, w \rangle$	4 : check $w' = w \wedge T < T_0$
5 : $S \Rightarrow: \langle \text{rnd}_s, r \rangle$	5 : $s \leftarrow \text{Pad}(\text{chal}, w)^{d_1''} \cdot \text{resp} \bmod n_1$
6 : check $c_{\text{rnd}} = \text{Enc}_{\text{pk}_{\text{RS}}}(\text{rnd}_s, r)$	6 : if $s^e \not\equiv \text{Pad}(\text{chal}, w) \pmod{n_1}$ then
7 : $\text{rnd}_A \leftarrow F(u', \text{PIN}) \oplus \text{rnd}_s$	7 : $T := T + 1$ and stop
8 : $m \leftarrow \text{Pad}(\text{msg}, F'(\text{msg}, \text{rnd}_A))$	8 : $\Rightarrow C : \langle \text{rnd}_s, r \rangle$
9 : $y \leftarrow m^{d_1'} \bmod n_1$	9 : $C \Rightarrow: \langle \text{msg}, y \rangle$
10 : $\Rightarrow S : \langle \text{msg}, y \rangle$	10 : $m \leftarrow \text{Pad}(\text{msg}, F'(\text{msg}, \text{rnd}_A))$
11 : $S \Rightarrow: \langle \sigma, w' \rangle$	11 : $\sigma_1 \leftarrow m^{d_1''} \cdot y \bmod n_1$
12 : $w := w'$	12 : if $\sigma_1^e \not\equiv m \pmod{n_1}$ then
13 : return σ	13 : $T := T + 1$ and stop
	14 : $\sigma \leftarrow \text{CRT}_{n_1, n_2}(\sigma_1, m^{d_2} \bmod n_2)$
	15 : update $w \leftarrow_{\$} \{0, 1\}^k$
	16 : $T := 0$
	17 : $C \Rightarrow: \langle \sigma, w \rangle$

Fig. 7. Electrum fail-stop server-supported signing protocol

Revoke _C ($\{(\text{msg}_i^*, \sigma_i^*)\}_{i \in \{1,2\}}, \text{PIN}\rangle$)	Verify-proof _{RS} (sk_{RS})
1 : $\text{rnd}_c \leftarrow F(u', \text{PIN})$	1 : $C := \langle \{(\text{msg}_i^*, \sigma_i^*)\}_{i \in \{1,2\}}, c_\pi \rangle$
2 : $\pi \leftarrow (c_{\text{rnd}}, \text{rnd}_c, n, e)$	2 : $(c_{\text{rnd}}, \text{rnd}_c, n, e) \leftarrow \text{Dec}(\text{sk}_{\text{RS}}, c_\pi)$
3 : $c_\pi \leftarrow \text{Enc}_{\text{pk}_{\text{RS}}}(\pi)$	3 : $\text{rnd}_A \leftarrow \text{rnd}_c \oplus \text{Dec}(\text{sk}_{\text{RS}}, c_{\text{rnd}})$
4 : $\Rightarrow RS : \langle \{(\text{msg}_i^*, \sigma_i^*)\}_{i \in \{1,2\}}, c_\pi \rangle$	4 : for $i = 1..2$ do
	5 : $b_i^* \leftarrow \text{Verify}((n, e), \text{msg}_i^*, \sigma_i^*)$
	6 : $m_i \leftarrow \text{Pad}(\text{msg}_i^*, F'(\text{msg}_i^*, \text{rnd}_A))$
	7 : $b_i^r \leftarrow [(\sigma_i^*)^e \bmod n = m_i]$
	8 : return $b_1^* \wedge b_2^* \wedge (b_1^r \oplus b_2^r)$

Fig. 8. Electrum revocation protocol

The execution of *Sys* must begin with P_3 submitting the (init)-command to $\mathcal{M}_3$. In response, $\mathcal{M}_3$ generates the keypair $(\text{pk}_{\text{RS}}, \text{sk}_{\text{RS}})$ for quantum-safe public-key encryption, and sends the *revocation service public key* pk_{RS} to $\mathcal{M}_1$, $\mathcal{M}_2$, and the adversary $\mathcal{A}$. The execution continues with the key generation initiated by P_1 and P_2 (Fig. 6), in parallel of which the adversary may corrupt the machines $\mathcal{M}_i$ (except $\mathcal{M}_3$). Key generation works as discussed in Sec. 2.1, with one important addition: the random seed $\text{rnd}_A \leftarrow \text{rnd}_c \oplus \text{rnd}_s$ for paddings is generated and secret-shared, where the client holds only the encryption u' of rnd_c and the commitment / encryption c_{rnd} of rnd_s . The machines $\mathcal{M}_1$ and $\mathcal{M}_2$ send the resulting public key pk to all other $\mathcal{M}_i$ and to $\mathcal{A}$.

The “*main*” part of the signing protocol (lines 2&8–13 in Sign_C , lines 9–17 in Sign_S) is the same as in Sec. 2.1, with the additional detail that the randomness used in the PSS padding [29, Sec. 9.1] of the message is generated from the seed rnd_A (used as the key of the PRP F'), as well as the message itself (used as the argument of F'). But before these steps, the client must learn rnd_s , for which it has the commitment c_{rnd} . The server opens that commitment to the client, but only after the client has authenticated itself by showing that it knows the exponent d'_1 . This creates an additional round-trip compared to [14,46].

The revocation protocol consists of the client collecting the evidence that one of the signatures σ_i^* of the message msg_i^* is a forgery, and the Revocation Service checking that evidence. The evidence consists of the random seed rnd_c and rnd_s . RS makes sure that both σ_1^* and σ_2^* pass verification, while exactly one of them is created from the given seed. These checks turn out to be sufficient to implement Fig. 5.

The adversary $\mathcal{A}$ may corrupt the machines $\mathcal{M}_i$. The corruptions are handled identically to Snetkov et al. [46], i.e. the machines may be corrupted at any time during the execution, resulting in $\mathcal{M}_i$ handing over its internal state and the control over its future steps to $\mathcal{A}$. Exceptionally, as discussed above, $\mathcal{M}_3$ cannot be corrupted. The corruption of $\mathcal{M}_1$ is more fine-grained, and may increase during the execution of the system. Besides *full control*, the adversary may *learn client’s encrypted memory*, netting him the values stored at the end of the key

generation protocol. Or, the adversary may *learn client's unencrypted memory*, additionally obtaining d'_1 and rnd_c .

As usual in UC setting, the adversary controls a machine $\mathcal{M}_i$ that it has taken over; this control is realized by $\mathcal{A}$ sending commands to that machine to send messages to other machines or the environment. Besides taking over machines, the adversary is also allowed to “masquerade” $\mathcal{M}_1$ in Fig. 7–8, i.e. $\mathcal{M}_2$ and $\mathcal{M}_3$ may participate in the signing or revocation protocol also with the adversary. This offers a way for the adversary to use the content of client's memory it has learned through a leak; this “masquerading” is part of the threat model of server-supported signatures.

5 Security

Proof Intuition and Explanation Unlike in many previous works on fail-stop signatures, we do not assume an unbounded adversary. As we described in Sec. 2.2, the external adversary has access to the factorization oracle, but even this oracle does not help it to find collisions of the hash function or break a PRP.

The idea behind our proof goes as follows: we construct the simulator Sim ensuring that no $\mathcal{Z}$ can distinguish the executions in the configurations $\mathcal{Z} \parallel \text{Sys} \parallel \mathcal{A} \parallel \mathcal{F}^{\text{helper}}$ and $\mathcal{Z} \parallel \mathcal{F}^{\text{SplFS}} \parallel \text{Sim} \parallel \mathcal{A} \parallel \mathcal{F}^{\text{helper}}$ for any adversary $\mathcal{A}$, where Sys is the composition of the machines $\mathcal{M}_1, \dots, \mathcal{M}_n$ executing the Electrum protocols as described in previous section.

The simulator simulates the machine $\mathcal{M}_3$ that $\mathcal{A}$ cannot corrupt; the simulation begins with the generation of the keys $\text{pk}_{\text{RS}}, \text{sk}_{\text{RS}}$ and the distribution of the former. The simulation continues with the key generation, possibly preceded with the corruption of either the client or the server. If neither are corrupted during key generation, then the simulator generates the moduli n_1, n_2 , the private exponents d_1, d_2 , and the padding seed rnd_A (and some other values). However, simulator does not yet secret-share generated values and does not generate the “encrypted memory” u, u' . Only when the adversary corrupts the server or requests access to client's memory, does the simulator generate the shares. The ideal functionality and the simulator are set up so that when the parties are corrupted by $\mathcal{A}$, then Sim obtains PIN in due time to make sure that u and d'_1 , and u' and rnd_c are correctly related.

If one of the parties is already corrupted during the key generation, then the simulator will participate in the key generation protocol as the honest party. If the other party also gets corrupted, then the simulator can reveal its internal state to the adversary.

Signing can be similarly simulated. If neither party is corrupted, then the simulator uses n_1, n_2, d_1, d_2 to construct the signature. If the adversary is masquerading the client, then the simulator can brute-force for the PIN' that the adversary is using by comparing the signed response (line 3 in Sign_C) with the challenge signed with private exponents generated from u using all possible PINs. If one of the parties is under full control of the adversary ($c_P = 3$ or $c_S = 1$), then the simulator can simulate the other party.

The verifications of signatures do not involve the simulator. We have to argue that the real system (that always executes the verification algorithm Ver) will give the same result as $\mathcal{F}^{\text{SplFS}}$. The difference can only stem from line 2 of Verification in Fig. 3, with $\mathcal{F}^{\text{SplFS}}$ returning 0 while the real system returns 1. This implies that the adversary has independently generated a message-signature pair (M, σ) without corrupting the parties to leak the private key, and without access to $\mathfrak{X}$.

Forgery proofs are simulated by the simulator creating the description of an algorithm $\mathfrak{A}$ that basically runs the Verify-proof_{RS} algorithm (Fig. 8). The values in the forgery proof come from the internal state of the simulator; some of the values (e.g. rnd_c) may be computed from the internal state and the PIN' that the client entered into the forgery proof generation. Indeed, while the simulator may not have the correct PIN and the used PIN' available to it, the algorithm $\mathfrak{A}$ gets to use them. Hence the execution of $\mathfrak{A}$ is in general the same as the execution of $\mathcal{M}_1$ and $\mathcal{M}_3$ in the real execution, also resulting in the same output. The only difference is $c_P = 0$, in which case the simulator has no u' . But even in this case, $\mathfrak{A}$ can check whether the client used the correct PIN and will be able to figure out whether the proof would have been accepted in a real execution.

We proceed to argue that $\mathfrak{A}$ returns the same result b_{adv} as $\mathcal{F}^{\text{SplFS}}$ also in cases where the output of $\mathcal{F}^{\text{SplFS}}$ is not taken as the value of b_{adv} . If $c_1 = 0$ or $c_2 = 0$ then the verification of σ_1 or σ_2 fails (because RSA-PSS is UF-CMA secure while $\mathfrak{X}$ is unavailable) and $\mathfrak{A}$ returns 0. If $c_1 = c_2 = 3$ then σ_1 and σ_2 were constructed before $\mathfrak{X}$ was available, and, due to the sUF-CMA security of RSA-PSS [32], must have been constructed during a call to the signing protocol, which means that their paddings were generated from the seed rnd_A . The real functionality is able to output 1 only if the adversary managed to find a different seed rnd'_A for one of the paddings; but this violates the 2nd preimage security of F' (with fixed plaintext). If $c_1 = 2$ and $c_2 = 3$ (or vice versa), then σ_2 was created before and σ_1 after $\mathfrak{X}$ became available, but before the adversary learned rnd_A . In this case, the proof will go through if the client uses the correct PIN to create it. All other cases (for c_1, c_2) will fall through to b_{adv} from $\mathfrak{A}$.

Theorem 4. *If RSA with padding function Pad is sUF-CMA secure and F, F', F_n are pseudo-random permutations even with access to the potentially superpolynomial functionality $\mathfrak{X}$ (for factorization), then the protocol presented in Sec. 4 is a secure implementation of $\mathcal{F}^{\text{SplFS}}$ in the $\mathcal{F}^{\text{helper}}$ -relativized model.*

Proof. The protocols in Figures 6–8 together with the discussion in Sec. 4 have defined us a real system Sys . In order to show that it is at least as secure as $\mathcal{F}^{\text{SplFS}}$, we will construct a simulator Sim , such that for any adversary $\mathcal{A}$, no environment $\mathcal{Z}$ can distinguish an execution with $\text{Sys} \parallel \mathcal{A} \parallel \mathcal{F}^{\text{helper}}$ from an execution with $\mathcal{F}^{\text{SplFS}} \parallel \text{Sim} \parallel \mathcal{A} \parallel \mathcal{F}^{\text{helper}}$. The state of the Simulator contains the values mentioned in the description of the ideal functionality and the protocols, namely:

$$\begin{aligned} & c_S, c_P, e, n, n_1, n_2, d_1, d'_1, d''_1, d_2, u, u', w, \text{rnd}_c, \text{rnd}_s, \text{rnd}_A, \\ & \text{pk}_{RS}, \text{sk}_{RS}, c_{\text{rnd}}, c_d, r, T, \text{PIN}, \text{pk}_{RS}, \text{sk}_{RS} . \end{aligned}$$

Most of these values are undefined at the start of an execution. Actually, only $c_P := 0$ and $c_S := 0$ are defined at the beginning, and the values pk_{RS} , sk_{RS} and e are defined during the initialization. Other values become defined while simulating key generation and also when the corruption level of the client or the server increases. Tables 1–3 give an overview of when each of these other values is initialized. In these tables, the values in the left / bottom cell are initialized during key generation. The rest are initialized when the corruption level of the client or the server increases. Recall that the case $c_P = c_S = 1$ is impossible.

Below, we outline the manner in which the simulator interacts with and reacts to commands from $\mathcal{F}^{SpIFS}$.

	$c_S = 0$	$c_S = 1$
$c_P \geq 2$	PIN, d'_1 , d''_1 , rnd_c , rnd_s	(empty)
$c_P = 1$	u , u'	NA. Go to $c_P \geq 2$
$c_P = 0$	n , n_1 , $\phi(n_1)$, n_2 , d_1 , d_2 , w , rnd_A , c_{rnd} , c_d , r , T	d'_1 , d''_1 , rnd_s , rnd_c

Table 1. The values that are defined in the simulator, if both the server and the client were uncorrupted during key generation. For each (c_P, c_S) value, the cells to the left and below are accounted for, too.

$c_P \geq 2$	PIN, u , u'
$c_P = 1$	NA. Go to $c_P \geq 2$
$c_P = 0$	n , n_1 , $\phi(n_1)$, n_2 , d_1 , d'_1 , d''_1 , rnd_c , rnd_s , rnd_A , w , c_{rnd} , c_d , r , T

Table 2. The values that are defined in the simulator, if the server was corrupted during key generation

$c_S = 0$	$c_S = 1$
n , n_1 , n_2 , d_2 , d''_1 , rnd_c , rnd_s , rnd_A , w , T , PIN	(empty)

Table 3. The values that are defined in the simulator, if the client was corrupted during key generation

5.1 Initialization

When $\mathcal{F}^{SpIFS}$ notifies the simulator that the initialization is taking place, generate revocation service keypair (pk_{RS}, sk_{RS}) , fix public exponent e , then set counter $T = 0$. Send pk_{RS} to $\mathcal{A}$.

5.2 Corruption before key generation

On command “Corrupt server” from $\mathcal{A}$ before the key generation has started, send (corrupt-server) to $\mathcal{F}^{\text{SplFS}}$.

On command “Corrupt client” from $\mathcal{A}$ before the key generation has started, send (corrupt-client, 3) to $\mathcal{F}^{\text{SplFS}}$.

5.3 Key generation

On command (keygen-init) from $\mathcal{F}^{\text{SplFS}}$, the simulator proceeds as follows; depending on the corruption level of the client and server.

Neither is corrupted. Simulate the execution of both client and server inside Sim , running the distributed key generation protocol in Fig. 6. During the execution, initialize only the values named in the cell $c_P = c_S = 0$ in table 1; the outputs of the key generation protocol can be computed from these values. In this case, the real protocol execution is identical to the simulated one, since all values are generated in the same way as in the real protocol. Send (key, Ver) back to $\mathcal{F}^{\text{SplFS}}$, where Ver is the description of the RSA-PSS verification algorithm using the public key just generated.

Client is corrupted. Simulate the server’s execution of the protocol, defining the values in the “ $c_S = 0$ ” cell of Table 3. Generate RSA modulus n_2 , compute private exponent $d_2 = e^{-1} \bmod \phi(n_2)$. Upon receiving $\langle n_1, d_1'', \text{rnd}_c \rangle$ from $\mathcal{A}$ (line 9 in KeyGen_S), compute $n = n_1 \cdot n_2$, sample clone detection bitstring $w \leftarrow \{0, 1\}^*$ and randomness $\text{rnd}_s \leftarrow \{0, 1\}^*$. Then, encrypt rnd_s as $\text{Enc}_{\text{pk}_{\text{RS}}}(\text{rnd}_s; r)$. Send $\langle n, w, c_{\text{rnd}}, \text{rnd}_s, r \rangle$ to $\mathcal{A}$.

Simulate a run of the signing protocol as the server (see App. 5.5 below, for client corrupted to level $c_P = 3$).

Let Ver be the description of the RSA-PSS verification algorithm using the public key just generated. Send (key, Ver) back to $\mathcal{F}^{\text{SplFS}}$.

The key generation with corrupted client triggers the leakage of client’s PIN. Hence receive (corrupt-pin, PIN) from $\mathcal{F}^{\text{SplFS}}$.

The real protocol execution is identical to the simulated one, since all values are generated in the same way as in the real protocol.

Server is corrupted. Simulate client’s execution of the protocol, defining the values in the “ $c_P = 0$ ” cell of Table 2. Generate RSA modulus n_1 , compute private $d_1 = e^{-1} \bmod \phi(n_1)$ and sample randomly $d_1'' \leftarrow \mathbb{Z}_{\phi(n_1)}$ and $\text{rnd}_c \leftarrow \{0, 1\}^*$. Send $\langle d_1'', n_1, \text{rnd}_c \rangle$ to $\mathcal{A}$. Upon receiving $\langle n, w, c_{\text{rnd}}, \text{rnd}_s, r \rangle$ from $\mathcal{A}$, check whether $n_1 \mid n$ and $\text{Enc}_{\text{pk}_{\text{RS}}}(\text{rnd}_s; r) = c_{\text{rnd}}$.

Simulate a run of the signing protocol as the client (see App. 5.5 below, for server corrupted to level $c_S = 1$).

Let Ver be the description of the RSA-PSS verification algorithm using the public key just generated. Send (key, Ver) back to $\mathcal{F}^{\text{SplFS}}$.

The real protocol execution is different from the simulated one in the way how d_1'' is generated. In the real protocol, $d_1'' = d_1 - d_1' \pmod{\phi(n_1)}$, where $d_1 = e^{-1} \pmod{\phi(n_1)}$ and $d_1' = F_{n_1}(u, \text{PIN})$. In the simulated, $d_1'' \leftarrow \mathbb{Z}_{\phi(n_1)}$. Since F_{n_1} is a bijection, the distribution of the value d_1'' in both cases is identical.

Both parties are corrupted. In this case, the adversary $\mathcal{A}$ runs the key generation protocol. At the end of the protocol, the adversary tells *Sim* what the machines $\mathcal{M}_1$ and $\mathcal{M}_2$ should output to the environment. The simulator extracts the public key pk from this command, defines Ver as the description of the RSA-PSS verification algorithm using pk , and sends (key, Ver) back to $\mathcal{F}^{\text{SplFS}}$.

5.4 Corruption after key generation

Corrupting the server. On command “Corrupt server” from $\mathcal{A}$, presumably to $\mathcal{M}_2$, after the keys have been generated, send (corrupt-server) to $\mathcal{F}^{\text{SplFS}}$ and proceed as follows:

- If $c_P = 0$, then we move from cell $(c_P, c_S) = (0, 0)$ to cell $(0, 1)$ in Table 1. Generate random $d_1', d_1'', \text{rnd}_c, \text{rnd}_s$, subject to $(d_1' + d_1'') \pmod{\phi(n_1)} = d_1$ and $\text{rnd}_c \oplus \text{rnd}_s = \text{rnd}_A$. Send the state of the server to $\mathcal{A}$.
- If $c_P = 1$, then we move from cell $(c_P, c_S) = (1, 0)$ to cell $(2, 1)$ in Table 1. I.e., $\mathcal{F}^{\text{SplFS}}$ sets $c_P := 2$ and the simulator receives $(\text{corrupt-pin}, \text{PIN})$ from $\mathcal{F}^{\text{SplFS}}$. Compute $d_1' \leftarrow F_{n_1}(u, \text{PIN})$, $\text{rnd}_c \leftarrow F(u', \text{PIN})$, $d_1'' \leftarrow (d_1 - d_1') \pmod{\phi(n_1)}$, and $\text{rnd}_s \leftarrow \text{rnd}_A \oplus \text{rnd}_c$. Send the state of the server to $\mathcal{A}$.
- If $c_P \geq 2$, then we move from cell $(c_P, c_S) = (2, 0)$ to cell $(2, 1)$ in Table 1, or from cell $c_S = 0$ to cell 1 in Table 3. All computations of the values in the internal state of the simulator have already been done. Send the state of the server to $\mathcal{A}$.

Leaking client’s encrypted memory. On command “Leak encrypted memory” from $\mathcal{A}$, presumably to $\mathcal{M}_1$, ignore this query if key generation has not yet taken place. Otherwise, send $(\text{corrupt-client}, 1)$ to $\mathcal{F}^{\text{SplFS}}$ and proceed as follows:

- If $c_S = 0$, then we move from cell $(c_P, c_S) = (0, 0)$ to cell $(1, 0)$ in Table 1. Generate random values u, u' . Send the values in client’s encrypted memory (the simulator has now generated all of them) to $\mathcal{A}$.
- If $c_S = 1$, then we move from cell $(c_P, c_S) = (0, 1)$ to cell $(1, 1)$ and then immediately to cell $(2, 1)$ in Table 1. Alternatively, we move from cell $c_P = 0$ to cell 1 and then to cell 2 in Table 2. The functionality $\mathcal{F}^{\text{SplFS}}$ sets $c_P := 2$ and the simulator receives $(\text{corrupt-pin}, \text{PIN})$ from $\mathcal{F}^{\text{SplFS}}$. Using that PIN, compute u and u' from the already computed values d_1' and rnd_c . Send the values in client’s encrypted memory to $\mathcal{A}$. (The adversary itself will find the values in client’s unencrypted memory)

Leaking client’s unencrypted memory with no previous leaks. On command “Leak unencrypted memory” from $\mathcal{A}$, presumably to $\mathcal{M}_1$, with $c_P = 0$, ignore this query if key generation has not yet taken place. Otherwise, send $(\text{corrupt-client}, 2)$ to $\mathcal{F}^{\text{SplFS}}$. The functionality $\mathcal{F}^{\text{SplFS}}$ sets $c_P := 2$ and the simulator receives $(\text{corrupt-pin}, \text{PIN})$ from $\mathcal{F}^{\text{SplFS}}$. Proceed as follows:

- If server has not been corrupted, then we move from cell $(c_P, c_S) = (0, 0)$ to the cell $(2, 0)$ in Table 1. The simulator generates random u, u' and uses PIN to construct the values d'_1, rnd_c from them. Send client’s unencrypted memory (values stored in line 11 of KeyGen_C) and the values d'_1, rnd_c to the adversary.
- If server has been corrupted, then we move from cell $(c_P, c_S) = (0, 1)$ to the cell $(2, 1)$ in Table 1. Use the PIN to compute u and u' from the previously initialized d'_1 and rnd_c by inverting F_{n_1} , and F . Send $\langle d'_1, u, u' \rangle$ to the $\mathcal{A}$.

Leaking client’s unencrypted memory, when encrypted memory has already leaked. On command “Leak unencrypted memory” from $\mathcal{A}$, presumably to $\mathcal{M}_1$, with $c_P = 0$, the keys have already been generated, and the server has not been corrupted. Send $(\text{corrupt-client}, 2)$ to $\mathcal{F}^{\text{SplFS}}$. The functionality $\mathcal{F}^{\text{SplFS}}$ sets $c_P := 2$ and the simulator receives $(\text{corrupt-pin}, \text{PIN})$ from $\mathcal{F}^{\text{SplFS}}$. We move from cell $(c_P, c_S) = (1, 0)$ to the cell $(2, 0)$ in Table 1. Using the PIN, compute d'_1 and rnd_c from u and u' , and send them to the adversary.

Fully corrupting the client. On command “Corrupt client” from $\mathcal{A}$, presumably to $\mathcal{M}_1$, send $(\text{corrupt-client}, 3)$ to $\mathcal{F}^{\text{SplFS}}$. If $c_P \leq 1$ before that command, then compute the state of the client and send it to the adversary in the same manner as in leaking client’s unencrypted memory. In any case, set $c_P := 3$ and expect $\mathcal{M}_1$ to be commanded by $\mathcal{A}$ in the rest of the execution.

5.5 Signing and verification

Signing by client and server. This corresponds to the client (part of $\mathcal{Z}$) sending the command $(\text{sign}, \text{sid}, M, \text{PIN}')$ to the system (either to $\mathcal{M}_1$ or $\mathcal{F}^{\text{SplFS}}$) and the server (part of $\mathcal{Z}$) sending the command $(\text{sign}, \text{sid})$ to the system (either to $\mathcal{M}_2$ or $\mathcal{F}^{\text{SplFS}}$). The simulator’s next steps depend on whether the client and the server have been corrupted (i.e. the control given to the adversary) or not.

Neither controlled by the adversary. The simulator learns of this run at the moment when it receives $(\text{sign-init}, \text{sid}, M)$ from $\mathcal{F}^{\text{SplFS}}$. The simulator uses rnd_A to pad the message M and n_1, n_2, d_1, d_2 to compute the signature σ . It sends $(\text{signature}, \text{sid}, \sigma)$ back the $\mathcal{F}^{\text{SplFS}}$.

Client controlled by the adversary. The simulator has received the forwarded $(\text{sign}, \text{sid}, M, \text{PIN}')$ command from $\mathcal{F}^{\text{SplFS}}$. The simulator tells $\mathcal{A}$ that client has sent this command to $\mathcal{M}_1$. The simulator participates in the signing protocol 7

with the adversary, using the values n_2 and d_2 (that it knows) to construct the second half of the signature. In the end, the simulator learns from $\mathcal{A}$ what $\mathcal{M}_1$ should return to the client. The simulator tells $\mathcal{F}^{\text{SplFS}}$ to return this value to the client.

Server controlled by the adversary. The simulator has received the forwarded $(\text{sign}, \text{sid})$ command from the ideal functionality, and also $(\text{sign-init}, \text{sid}, M)$ as the message from $\mathcal{F}^{\text{SplFS}}$. The simulator has the values n_1, d'_1, rnd_c that it needs to participate in a signing protocol with the adversary. The protocol results in a signature σ ; the simulator returns $(\text{signature}, \text{sid}, \sigma)$ to $\mathcal{F}^{\text{SplFS}}$.

Both controlled by the adversary. In this case, the simulator forwards the forwarded messages to the adversary, and learns from it the outputs that it tells $\mathcal{F}^{\text{SplFS}}$ to give to the client and the server.

Signing by client and adversary. This corresponds to the client (part of $\mathcal{Z}$) sending the command $(\text{sign}, \text{sid}, M, \text{PIN}')$ to the system (either to $\mathcal{M}_1$ or $\mathcal{F}^{\text{SplFS}}$), and the adversary $\mathcal{A}$ telling the corrupted $\mathcal{M}_2$ to run the signing protocol. In executions with $\mathcal{F}^{\text{SplFS}}$, the latter is received by the simulator who sends the command $(\text{sign-server}, \text{sid})$ to $\mathcal{F}^{\text{SplFS}}$.

The ideal functionality sends $(\text{sign-init}, \text{sid}, M)$ to the simulator. If the client is not under adversary's control, then the simulator has the values n_1, d'_1, rnd_c that it needs to run the signing protocol with the adversary. The protocol results in a signature σ ; the simulator returns $(\text{signature}, \text{sid}, \sigma)$ to $\mathcal{F}^{\text{SplFS}}$. If the client is also adversarially controlled, then $\mathcal{A}$ will tell what $\mathcal{F}^{\text{SplFS}}$ returns to the client.

Signing by the adversary and the server. This corresponds to the server (a part of $\mathcal{Z}$) sending the command $(\text{sign}, \text{sid})$ to $\mathcal{M}_2$ or $\mathcal{F}^{\text{SplFS}}$, while the adversary $\mathcal{A}$, masquerading as the client, contacts $\mathcal{M}_2$ (or the simulator), initiating a signing session (Fig. 7). In this setting, the server is not corrupted, otherwise the adversary would not need to masquerade to it. For the same reason, the client is not fully corrupted ($c_P = 3$).

In response to the initiation, the simulator (playing the honest $\mathcal{M}_2$) comes up with a challenge chal , and sends it to the adversary. The adversary sends back a value w' and $v \in \mathbb{Z}_{n_1}$, where supposedly $v \equiv \text{chal}^{d'_1} \pmod{n_1}$.

If $c_P = 0$, then the simulator sends $(\text{sign-client}, M, \text{PIN}_0)$ to $\mathcal{F}^{\text{SplFS}}$, where PIN_0 is some constant PIN. At this point, the execution stops, because $\mathcal{F}^{\text{SplFS}}$ will ignore this query. The same would have happened in the execution of the real system Sys , because the adversary would not have any information about d'_1 .

If $c_P \geq 1$, then the simulator has the value u . The simulator tries out all possible values of $\text{PIN}' \in \{0, \dots, L-1\}$, and finds the one for which $\text{chal}^{F(u, \text{PIN}')} \equiv v \pmod{n_1}$. The simulator sends $(\text{sign-client}, M, \text{PIN}')$ to $\mathcal{F}^{\text{SplFS}}$. If $\text{PIN}' \neq \text{PIN}$ then the execution again ends, this is the same effect as in the execution of the real system Sys .

If $\text{PIN}' = \text{PIN}$, then $c_P := 2$. The ideal functionality sends $(\text{sign-init}, \text{sid}, M)$ to the simulator. The simulator completes the run of signing protocol with the adversary, obtaining the signature σ , which it sends back to the ideal functionality.

Verification. The verification of signatures does not involve the simulator. However, we have to argue that the real system Sys (that always executes $\text{Ver}(\text{pk}, M, \sigma)$ at this point) returns the same verification outcomes as the ideal functionality (that also uses the table T_{sign} of verification results). We see that due to the internal logic of $\mathcal{F}^{\text{SplFS}}$, the only possible deviation is the real protocol returning 1 for some (M, σ) , while the ideal functionality returns 0 because no $T_{\text{sign}}(M, \sigma') = c$ has been recorded for any σ' and $c \geq 1$ and, moreover, $b_{\text{TC}} = 1$ and $b_{\text{sp}} = 1$. This corresponds to the construction of the pair (M, σ) without M being an argument to some signing query, without leaking the private key, and without access to the superpolynomial $\mathcal{F}^{\text{helper}}$.

5.6 Forgery proof

The simulation depends on the corruption level of the client, and on the initiator of the forgery proof. The arguments on why the simulation is indistinguishable from the real system also heavily depend on the values $c_1 = T_{\text{sign}}(M_1, \sigma_1)$ and $c_2 = T_{\text{sign}}(M_2, \sigma_2)$ for the signatures σ_1, σ_2 on the messages M_1, M_2 that are submitted to the forgery proof protocol, one as the signature to be revoked and the other as the proof of forgery.

Simulation with client corrupted to level 3. In this case, the ideal functionality forwards $(\text{revoke}, \text{sid}, M_1, \sigma_1, M_2, \sigma_2, \text{PIN}')$ to the simulator. Simulator tells $\mathcal{A}$ that the client (part of $\mathcal{Z}$) told $\mathcal{M}_1$ to initiate a revocation protocol session with the message $(\text{revoke}, \text{sid}, M_1, \sigma_1, M_2, \sigma_2, \text{PIN}')$. The adversary $\mathcal{A}$ comes up with a forgery proof $\text{Enc}_{\text{pk}_{\text{RS}}}(\pi_{\text{adv}})$ and tells the simulator that $\mathcal{M}_1$ should send this proof and $(M_1, \sigma_1, M_2, \sigma_2)$ to $\mathcal{M}_3$. The simulator decrypts the proof and runs the proof check on it, identically to RS in the revocation protocol (Fig. 8). Let b_{RS} be the result. Simulator sends to $\mathcal{F}^{\text{SplFS}}$ the description of an algorithm $\mathfrak{A}$ that ignores its inputs, but always replies b_{RS} .

We have to argue that the response from $\mathcal{F}^{\text{SplFS}}$ to RS always coincides with the response b_{RS} that $\mathcal{M}_3$ would have sent to RS. The only possibility for differing outputs is $c_1 \cdot c_2 = 0$ or $c_1 = c_2 = 3$, because otherwise $\mathcal{F}^{\text{SplFS}}$ assigns $b_{\text{out}} := b_{\text{adv}} := b_{\text{RS}}$ and returns this to RS (Fig. 5). If $c_1 \cdot c_2 = 0$ or $c_1 = c_2 = 3$ then the ideal functionality rejects the proof of forgery. Let us argue that in the real system, $\mathcal{M}_3$ must also reject that proof.

If some $c_i = 0$, then (Fig. 3) this means that σ_i does not pass the signature verification for M_i . Either $\text{Ver}(M_i, \sigma_i) = 0$, or $\mathcal{F}^{\text{SplFS}}$ saw (but did not cause the creation of) σ_i before the superpolynomial $\mathcal{F}^{\text{helper}}$ was switched on. In the latter case, due to UF-CMA security of RSA-PSS (before large numbers can

be factored), also $\text{Ver}(M_i, \sigma_i) = 0$. Hence the RS rejects the proof (line 5 in Verify-proof_{RS}).

If $c_1 = c_2 = 3$, then σ_1 and σ_2 pass verification, and they were both created before the superpolynomial functionality was available. The signature σ_i was either constructed by signing protocol (in real execution; via a signing call to $\mathcal{F}^{\text{SplFS}}$ in the ideal execution), or by modifying an existing signature. The latter is impossible due to the sUF-CMA security of RSA-PSS [32]¹⁰. Hence the paddings of σ_1 and σ_2 were originally generated from $F'(M_1, \text{rnd}_A)$ and $F'(M_2, \text{rnd}_A)$. In order to make RS accept, the adversary has to come up with $\text{rnd}'_A \neq \text{rnd}_A$ (which it would include in the forgery proof π), such that $F'(M_i, \text{rnd}_A) = F'(M_i, \text{rnd}'_A)$ for some $i \in \{1, 2\}$. If the adversary is successful, then it means that the PRP F' with a fixed plaintext (i.e. as function from the key to the ciphertext) is not second-preimage resistant. We have a good reason to believe that AES, when used like this, is second-preimage resistant, because its use to construct hash functions has been standardized [28].

Simulation with client corrupted to level 0–2. In this case, the revocation protocol may be initiated either by the client or by the adversary. If it is initiated by the client, then the simulator learns of it at the moment the ideal functionality sends it the message $(\text{revoke-init}, \text{sid}, M_1, \sigma_1, M_2, \sigma_2)$. If it is initiated by the adversary (masquerading as the client), then the simulator learns of it at the moment the adversary $\mathcal{A}$ sends it the message consisting of $M_1, \sigma_1, M_2, \sigma_2, c_\pi$ (line 4 in Revoke_C); the adversary sends this message presumably to $\mathcal{M}_3$, claiming that it came from $\mathcal{M}_1$. In the latter case, the simulator performs the following operations in order to determine PIN' that the adversary used:

- Decrypt $\pi \leftarrow \text{Dec}(\text{sk}_{RS}, c_\pi)$ and learn rnd'_c from it.
- If $c_P \geq 1$ then try out all L possible PIN' values and check whether $\text{rnd}'_c = F'(u', \text{PIN}')$;
 - If a match was found, then use this value of PIN' .
 - If no match was found, or if $c_P = 0$, then let $\text{PIN}' \leftarrow L$ (i.e. outside the normal range).

Continuing the processing of adversary-initiated revocation protocol: the simulator sends $(\text{revoke}, \text{sid}, M_1, \sigma_1, M_2, \sigma_2, \text{PIN}')$ to the ideal functionality, and the simulator sends back $(\text{revoke-init}, \text{sid}, M_1, \sigma_1, M_2, \sigma_2)$.

In both cases, the simulator will prepare an algorithm $\mathfrak{A}$ that takes as arguments two PINs — PIN and PIN' , and then works as shown in Fig. 9. All values except PIN and PIN' are hardcoded into $\mathfrak{A}$. The simulator sends $(\text{revoke-alg}, \text{sid}, \mathfrak{A})$ to the ideal functionality.

¹⁰ We are appealing to the sUF-CMA security of RSA-PSS here. In Fig. 3, the logic of filling T_{sign} corresponds to a signature scheme that is only UF-CMA-secure. We could have assumed / required sUF-CMA security there as well, thereby probably simplifying the logic. We choose to keep the current logic, to be able to support UF-CMA-secure signature schemes, where some extra property of the padding ensures that one cannot change the seed from which it is generated.

If $c_P = 0$	If $c_P \in \{1, 2\}$
1 : $b_{\text{PIN}} \leftarrow [\text{PIN} = \text{PIN}']$	1 : $r_c \leftarrow F'(u', \text{PIN})$
2 : $b_1^* \leftarrow \text{Verify}((n, e), M_1, \sigma_1)$	2 : $r'_c \leftarrow F(u', \text{PIN}')$
3 : $m_1 \leftarrow \text{Pad}(M_1, F'(M_1, \text{rnd}_A))$	3 : $r_A \leftarrow \text{rnd}_A \oplus r_c \oplus r'_c$
4 : $b_1^r \leftarrow [\sigma_1^e \bmod n = m_1]$	4 : $m_1 \leftarrow \text{Pad}(M_1, F'(M_1, r_A))$
5 : $b_2^* \leftarrow \text{Verify}((n, e), M_2, \sigma_2)$	5 : $b_1^r \leftarrow [\sigma_1^e \bmod n = m_1]$
6 : $m_2 \leftarrow \text{Pad}(M_2, F'(M_2, \text{rnd}_A))$	6 : $b_2^* \leftarrow \text{Verify}((n, e), M_2, \sigma_2)$
7 : $b_2^r \leftarrow [\sigma_2^e \bmod n = m_2]$	7 : $m_2 \leftarrow \text{Pad}(M_2, F'(M_2, r_A))$
8 : return $b_{\text{PIN}} \wedge b_1^* \wedge b_2^* \wedge (b_1^r \oplus b_2^r)$	8 : $b_2^r \leftarrow [\sigma_2^e \bmod n = m_2]$
	9 : return $b_1^* \wedge b_2^* \wedge (b_1^r \oplus b_2^r)$

Fig. 9. Simulator's algorithm $\mathfrak{A}(\text{PIN}, \text{PIN}')$, if client is not fully corrupted

We again have to argue that the real system's output to the RS is the same as the output that $\mathcal{F}^{\text{SpIFS}}$ would give. If $c_P = c_S = 0$ and the revocation was initiated by the adversary, then $\mathcal{F}^{\text{SpIFS}}$ rejects. In this case, the proof c_π created by $\mathcal{A}$ won't pass the checks performed by $\mathcal{M}_3$ either, because $\mathcal{A}$ has no way to find out rnd_c and rnd_s .

The next check in Fig. 5 is the test whether $c_1 = 0$ or $c_2 = 0$. If it passes, then $\mathcal{F}^{\text{SpIFS}}$ returns 0 to the RS. In this case, as before, the verification of either σ_1 or σ_2 does not pass, meaning that in real system, $\mathcal{M}_3$ also returns 0 to RS.

If $c_1 = c_2 = 3$ then in the ideal execution, $\mathcal{F}^{\text{SpIFS}}$ returns 0 to the RS. In the real execution, the paddings in σ_1 and σ_2 are generated from the same random seed rnd_A , meaning that $\mathcal{M}_3$ will obtain $b_1^r = b_2^r$ and will thus also return 0 to the RS.

If $c_1 = 2$ and $c_2 = 3$ (the "vice versa" case is identical), then σ_2 was created before the superpolynomial helper was available, while σ_1 was created after the helper was available but before the adversary had the chance to learn rnd_A . In this case, in the real execution, the client can cause $\mathcal{M}_1$ to create a forgery proof accepted by $\mathcal{M}_3$, if it uses the correct $\text{PIN}' = \text{PIN}$. The algorithm $\mathfrak{A}$ tests whether $\text{PIN}' = \text{PIN}$, this test is effectively in place in both variants of $\mathfrak{A}$ in Fig. 9. The reason for having two variants of $\mathfrak{A}$ is, that if $c_P \geq 1$ and the revocation protocol was initiated by $\mathcal{A}$, then the adversary has some control over rnd_A that $\mathcal{M}_3$ will use in Verify-proof_{RS} . This level of control is reflected in $\mathfrak{A}$.

In the rest of the cases (for the values c_1 and c_2), the ideal functionality give the output of $\mathfrak{A}$ to the RS. In a real execution, the outcome of Verify-proof_{RS} is the same.

□

6 Implementation Details of Electrum

6.1 Parameters

We outline some parameter recommendations in Table 4 based on the current Smart-ID setup ¹¹. The parameters of actual systems could be updated based on known capabilities of classical and quantum adversaries.

Parameters	Electrum	Clarification
$ n_i $	3072 bits	Client/server modulus' size
$ n $	6144 bits	Total modulus' size
$ u $	128 bits	Client's private exponent bitstring's size
$ u' $	128 bits	Client's randomness bitstring's size
$ PIN $	10^6	Number of possible PIN codes
$ rnd_i $	128 bits	Client/server randomness' size
$F / F' / F_{n_1}$	AES-128 based	Pseudo-random permutations
Pad	RSA-PSS	Padding method

Table 4. Parameters for Electrum protocol

6.2 Organizational Issues of Revocation Scenarios

In this section, we discuss organisational issues which could occur after revocation of the signing key in Electrum protocol by client. We provide ideas and recommendations on how to resolve some of them but leave legal/organisation specifics to future work.

Revocation rights For Electrum protocol, we assume existence of Revocation Service which could be provided either by certificate authority or government body. Revocation of signing keys would not be changed drastically compared to existing account closing procedure ¹². However, after a user has shown to RS that someone forged their signature(s), legal question arises: who and how should retract forged signature(s)? We leave this question to regulations such as eIDAS ¹³ and judicial practices of countries where Electrum protocol would be implemented.

¹¹ <https://github.com/SK-EID/smart-id-documentation/wiki>

¹² <https://www.smart-id.com/help/faq/closing-the-account/closing-smart-id-accounts/>

¹³ <https://www.eid.as/>

Signing cases For a client to find potential forgeries they either must store every signature produced by **Electrum** protocol or there must be a service which stores every generated signature. Such service must be as trustful as RS, since they could easily delete the track of any signature in the logs. Interestingly, similar *transaction tracking* service already exists for Smart-ID ¹⁴.

Authentication cases We mainly see the main use-case of **Electrum** protocol as signing service. However, the existing Smart-ID application could be used for both signing and authentication. For the cases of authentication, we cannot promise any security guarantees after malicious adversary authorised in a system (i.e banking) beyond a proof of malicious authentication. Track and retraction of adversarial actions falls down upon logging and administrative utilities of the affected system.

7 Conclusions

We have defined the **Electrum** protocol set, an interoperable replacement for the current Smart-ID protocols, where the interoperability means that **Electrum** signatures look identical to Smart-ID signatures, which in turn are verifiable by a standard RSA-PSS verifier. The way of constructing the paddings of to-be-signed messages gives the user/client the ability to show that a signature has been forged, even if the server does not cooperate. On the other hand, if the server is honest then the client is unable to create repudiable signatures. It is possible to upgrade the existing Smart-ID system to **Electrum**, i.e. update the client app and the server software, without affecting the rest of the ecosystem. Together with updating server software, the Revocation Service has to be introduced and its public encryption key distributed.

The ideal functionality $\mathcal{F}^{\text{SpIFS}}$ provides the minimal functionality for setting up a stopgap for a cryptographically relevant quantum computer (CRQC). The full system requires more components, for example a mechanism to distribute the decisions of the RS on whether a signature in a pair of signatures has been shown to be forged. In a full system, the client should also revoke its public key or at least refrain from creating any further signatures with it once it has created a forgery proof for one of the signatures verifiable with this public key (and revealed rnd_c and rnd_s to the RS in the process). For revoking forged signatures, the client needs to create and keep a signature whose genuineness is undisputable. All these additional functionalities can be part of a larger system around a secure implementation of $\mathcal{F}^{\text{SpIFS}}$.

Importantly, this larger system currently exists. It consists of the PKI and time-stamping services that are part of an authentication / digital signatures ecosystem. If the current Smart-ID protocols were replaced with **Electrum**, then these additional services would immediately perform the functions that a framework for fail-stop signatures needs.

¹⁴ <https://portal.smart-id.com/transactions>

Electrum includes the Revocation Service, both in the implementation as well as in the definition of $\mathcal{F}^{\text{SplFS}}$. An interesting future work would be the construction of a system with *public revocability*; both the ideal functionality and the protocol (which should still produce signatures looking like RSA-PSS signatures) would have to be modified for that.

8 Acknowledgements

This work was funded by the Estonian Research Council under the grant number PRG1780.

References

1. Alkadri, N.A., Döttling, N., Pu, S.: Practical Lattice-Based Distributed Signatures for a Small Number of Signers. In: Pöpper, C., Batina, L. (eds.) Applied Cryptography and Network Security. pp. 376–402. Springer Nature Switzerland, Cham (2024). https://doi.org/10.1007/978-3-031-54770-6_15
2. Alnahawi, N., Müller, J., Oupický, J., Wiesmaier, A.: A Comprehensive Survey on Post-Quantum TLS. IACR Communications in Cryptology **1**(2) (2024). <https://doi.org/10.62056/ahee0iuc>
3. Barić, N., Pfitzmann, B.: Collision-Free Accumulators and Fail-Stop Signature Schemes Without Trees. In: Fumy, W. (ed.) Advances in Cryptology — EURO-CRYPT '97. pp. 480–494. Springer Berlin Heidelberg, Berlin, Heidelberg (1997). https://doi.org/10.1007/3-540-69053-0_33
4. Bellare, M., Neven, G.: Multi-Signatures in the Plain Public-Key Model and a General Forking Lemma. In: Proceedings of the 13th ACM Conference on Computer and Communications Security. p. 390–399. CCS '06, Association for Computing Machinery, New York, NY, USA (2006). <https://doi.org/10.1145/1180405.1180453>
5. Bellare, M., Rogaway, P.: On the Construction of Variable-Input-Length Ciphers. In: Knudsen, L.R. (ed.) Fast Software Encryption, 6th International Workshop, FSE '99, Rome, Italy, March 24–26, 1999, Proceedings. Lecture Notes in Computer Science, vol. 1636, pp. 231–244. Springer (1999), https://doi.org/10.1007/3-540-48519-8_17
6. Bendlin, R., Krehbiel, S., Peikert, C.: How to Share a Lattice Trapdoor: Threshold Protocols for Signatures and (H)IBE. In: Jacobson, M., Locasto, M., Mohassel, P., Safavi-Naini, R. (eds.) Applied Cryptography and Network Security. pp. 218–236. Springer Berlin Heidelberg, Berlin, Heidelberg (2013). https://doi.org/10.1007/978-3-642-38980-1_14
7. Bernstein, D.J.: Cryptographic Competitions. Journal of Cryptology **37**(1), 7 (Dec 2023). <https://doi.org/10.1007/s00145-023-09467-1>
8. Bienstock, A., de Castro, L., Escudero, D., Polychroniadou, A., Takahashi, A.: Efficient, Scalable Threshold ML-DSA Signatures: An MPC Approach. Cryptology ePrint Archive, Paper 2025/1163 (2025), <https://eprint.iacr.org/2025/1163>
9. Black, J., Rogaway, P.: Ciphers with Arbitrary Finite Domains. In: Preneel, B. (ed.) Topics in Cryptology - CT-RSA 2002, The Cryptographer's Track at the RSA Conference, 2002, San Jose, CA, USA, February 18–22, 2002, Proceedings. Lecture Notes in Computer Science, vol. 2271, pp. 114–130. Springer (2002), https://doi.org/10.1007/3-540-45760-7_9

10. Boneh, D., Gennaro, R., Goldfeder, S., Jain, A., Kim, S., Rasmussen, P.M.R., Sahai, A.: Threshold Cryptosystems from Threshold Fully Homomorphic Encryption. In: Shacham, H., Boldyreva, A. (eds.) *Advances in Cryptology – CRYPTO 2018*. pp. 565–596. Springer International Publishing, Cham (2018). https://doi.org/10.1007/978-3-319-96884-1_19
11. Borin, G., Celi, S., del Pino, R., Espitau, T., Niot, G., Prest, T.: Threshold Signatures Reloaded: ML-DSA and Enhanced Raccoon with Identifiable Aborts. *Cryptology ePrint Archive*, Paper 2025/1166 (2025), <https://eprint.iacr.org/2025/1166>
12. Boschini, C., Dahari, H., Naor, M., Ronen, E.: That’s Not My Signature! Fail-Stop Signatures for a Post-quantum World. In: Reyzin, L., Stebila, D. (eds.) *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference*, Santa Barbara, CA, USA, August 18–22, 2024, *Proceedings, Part I. Lecture Notes in Computer Science*, vol. 14920, pp. 107–140. Springer (2024). https://doi.org/10.1007/978-3-031-68376-3_4
13. Boschini, C., Takahashi, A., Tibouchi, M.: MuSig-L: Lattice-Based Multi-signature with Single-Round Online Phase. In: Dodis, Y., Shrimpton, T. (eds.) *Advances in Cryptology – CRYPTO 2022*. pp. 276–305. Springer Nature Switzerland, Cham (2022). https://doi.org/10.1007/978-3-031-15979-4_10
14. Buldas, A., Kalu, A., Laud, P., Oruaas, M.: Server-supported RSA signatures for mobile devices. In: *Computer Security–ESORICS 2017: 22nd European Symposium on Research in Computer Security*, Oslo, Norway, September 11–15, 2017, *Proceedings, Part I*. pp. 315–333. Springer (2017). https://doi.org/10.1007/978-3-319-66402-6_19
15. Buser, M., Dowsley, R., Esgin, M., Gritti, C., Kasra Kermanshahi, S., Kuchta, V., Legrow, J., Liu, J., Phan, R., Sakzad, A., Steinfeld, R., Yu, J.: A Survey on Exotic Signatures for Post-quantum Blockchain: Challenges and Research Directions. *ACM Comput. Surv.* **55**(12) (Mar 2023). <https://doi.org/10.1145/3572771>
16. Canetti, R.: Universally Composable Security: A New Paradigm for Cryptographic Protocols. In: 42nd Annual Symposium on Foundations of Computer Science, FOCS 2001, 14–17 October 2001, Las Vegas, Nevada, USA. pp. 136–145. IEEE Computer Society (2001). <https://doi.org/10.1109/SFCS.2001.959888>
17. Chairattana-Apirom, R., Tessaro, S., Zhu, C.: Partially Non-interactive and Lattice-Based Threshold Signatures. In: Chung, K.M., Sasaki, Y. (eds.) *Advances in Cryptology – ASIACRYPT 2024*. pp. 268–302. Springer Nature Singapore, Singapore (2025). https://doi.org/10.1007/978-981-96-0894-2_9
18. Chen, Y.: DualMS: Efficient Lattice-Based und Multi-signature with Trapdoor-Free Simulation. In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology – CRYPTO 2023*. pp. 716–747. Springer Nature Switzerland, Cham (2023). https://doi.org/10.1007/978-3-031-38554-4_23
19. Cozzo, D., Smart, N.P.: Sharing the LUOV: Threshold Post-quantum Signatures. In: Albrecht, M. (ed.) *Cryptography and Coding – 17th IMA International Conference, IMACC 2019, Oxford, UK, December 16–18, 2019, Proceedings. Lecture Notes in Computer Science*, vol. 11929, pp. 128–153. Springer (2019). https://doi.org/10.1007/978-3-030-35199-1_7
20. Damgård, I., Orlandi, C., Takahashi, A., Tibouchi, M.: Two-Round n-out-of-n and Multi-Signatures and Trapdoor Commitment from Lattices. *Journal of Cryptology* **35**(2), 14 (Apr 2022). <https://doi.org/10.1007/s00145-022-09425-3>
21. Desmedt, Y., Frankel, Y.: Threshold cryptosystems. In: Brassard, G. (ed.) *Advances in Cryptology — CRYPTO’ 89 Proceedings*. pp. 307–315. Springer New York, New York, NY (1990). https://doi.org/10.1007/0-387-34805-0_28

22. Dufka, A., Kravtšenko, S., Laud, P., Snetkov, N.: Trilithium: Efficient and Universally Composable Distributed ML-DSA Signing. *Cryptology ePrint Archive*, Paper 2025/675 (2025), <https://eprint.iacr.org/2025/675>
23. Espitau, T., Katsumata, S., Takemure, K.: Two-Round Threshold Signature from Algebraic One-More Learning with Errors. In: Reyzin, L., Stebila, D. (eds.) *Advances in Cryptology – CRYPTO 2024*. pp. 387–424. Springer Nature Switzerland, Cham (2024). https://doi.org/10.1007/978-3-031-68394-7_13
24. Fleischhacker, N., Herold, G., Simkin, M., Zhang, Z.: Chipmunk: Better Synchronized Multi-Signatures from Lattices. In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. p. 386–400. CCS '23, Association for Computing Machinery, New York, NY, USA (2023). <https://doi.org/10.1145/3576915.3623219>
25. Fleischhacker, N., Simkin, M., Zhang, Z.: Squirrel: Efficient Synchronized Multi-Signatures from Lattices. In: *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. p. 1109–1123. CCS '22, Association for Computing Machinery, New York, NY, USA (2022). <https://doi.org/10.1145/3548606.3560655>
26. Gur, K.D., Katz, J., Silde, T.: Two-Round Threshold Lattice-Based Signatures from Threshold Homomorphic Encryption. In: Saarinen, M.J., Smith-Tone, D. (eds.) *Post-Quantum Cryptography*. Springer Nature Switzerland, Cham (2024). https://doi.org/10.1007/978-3-031-62746-0_12
27. Hinek, M.J.: On the security of multi-prime RSA. *Journal of Mathematical Cryptology* **2**(2), 117–147 (2008). <https://doi.org/10.1515/JMC.2008.006>
28. Information technology - Security techniques - Hash-functions - Part 2: Hash-functions using an n-bit block cipher (2010), international Standard ISO/IEC 10118-2
29. Jonsson, J., Kaliski, B.: Public-Key Cryptography Standards (PKCS) #1: RSA Cryptography Specifications Version 2.1. RFC 3447 (Feb 2003). <https://doi.org/10.17487/RFC3447>
30. Kampanakis, P., Lepoint, T.: Vision Paper: Do We Need to Change Some Things? In: Günther, F., Hesse, J. (eds.) *Security Standardisation Research*. pp. 78–102. Springer Nature Switzerland, Cham (2023). https://doi.org/10.1007/978-3-031-30731-7_4
31. Katsumata, S., Reichle, M., Takemure, K.: Adaptively Secure 5 Round Threshold Signatures from MLWE/MSIS and DL with Rewinding. In: Reyzin, L., Stebila, D. (eds.) *Advances in Cryptology – CRYPTO 2024*. pp. 459–491. Springer Nature Switzerland, Cham (2024). https://doi.org/10.1007/978-3-031-68394-7_15
32. Lysyanskaya, A.: Security Analysis of RSA-BSSA. In: Boldyreva, A., Kolesnikov, V. (eds.) *Public-Key Cryptography - PKC 2023 - 26th IACR International Conference on Practice and Theory of Public-Key Cryptography*, Atlanta, GA, USA, May 7–10, 2023, *Proceedings, Part I*. Lecture Notes in Computer Science, vol. 13940, pp. 251–280. Springer (2023), https://doi.org/10.1007/978-3-031-31368-4_10
33. National Institute of Standards and Technology: Module-Lattice-Based Digital Signature Standard (aug 2024). <https://doi.org/10.6028/nist.fips.204>
34. National Institute of Standards and Technology: Module-Lattice-Based Key-Encapsulation Mechanism Standard (Aug 2024). <https://doi.org/10.6028/nist.fips.203>
35. National Institute of Standards and Technology: Stateless Hash-Based Digital Signature Standard (Aug 2024). <https://doi.org/10.6028/nist.fips.205>

36. Nomura, M., Nakamura, K.: On Fail-Stop Signature Schemes with H -EUC Security. *IEICE Trans. Fundam. Electron. Commun. Comput. Sci.* **102-A**(1), 125–147 (2019). <https://doi.org/10.1587/TRANSFUN.E102.A.125>
37. Paquin, C., Stebila, D., Tamvada, G.: Benchmarking Post-quantum Cryptography in TLS. In: Ding, J., Tillich, J.P. (eds.) *Post-Quantum Cryptography*. pp. 72–91. Springer International Publishing, Cham (2020). https://doi.org/10.1007/978-3-030-44223-1_5
38. Pedersen, T.P., Pfitzmann, B.: Fail-Stop Signatures. *SIAM Journal on Computing* **26**(2), 291–330 (1997). <https://doi.org/10.1137/S009753979324557X>
39. Pfitzmann, B.: *Digital Signature Schemes*. Springer Berlin Heidelberg (1996). <https://doi.org/10.1007/bfb0024619>
40. Pfitzmann, B., Waidner, M.: *Formal Aspects of Fail-Stop Signatures*, vol. 22. Univ., Fak. für Informatik (1990)
41. Pfitzmann, B., Waidner, M.: A Model for Asynchronous Reactive Systems and its Application to Secure Message Transmission. In: *2001 IEEE Symposium on Security and Privacy*, Oakland, California, USA May 14–16, 2001. pp. 184–200. IEEE Computer Society (2001). <https://doi.org/10.1109/SECPRI.2001.924298>
42. del Pino, R., Espitau, T., Niot, G., Prest, T.: Simple and Efficient Lattice Threshold Signatures with Identifiable Aborts. *Cryptology ePrint Archive*, Paper 2025/871 (2025), <https://eprint.iacr.org/2025/871>
43. del Pino, R., Katsumata, S., Maller, M., Mouhartem, F., Prest, T., Saarinen, M.J.: Threshold Raccoon: Practical Threshold Signatures from Standard Lattice Assumptions. In: Joye, M., Leander, G. (eds.) *Advances in Cryptology – EUROCRYPT 2024*. pp. 219–248. Springer Nature Switzerland, Cham (2024). https://doi.org/10.1007/978-3-031-58723-8_8
44. del Pino, R., Niot, G.: Finally! A Compact Lattice-Based Threshold Signature. In: Jager, T., Pan, J. (eds.) *Public-Key Cryptography – PKC 2025*. pp. 169–199. Springer Nature Switzerland, Cham (2025). https://doi.org/10.1007/978-3-031-91826-1_6
45. Snetkov, N., Vakarjuk, J., Laud, P.: TOPCOAT: towards practical two-party Crystals-Dilithium. *Discover Computing* **27**(1), 18 (Jul 2024). <https://doi.org/10.1007/s10791-024-09449-2>
46. Snetkov, N., Vakarjuk, J., Laud, P.: Universally Composable Server-Supported Signatures for Smartphones. *Cryptology ePrint Archive*, Paper 2024/1941 (2024), <https://eprint.iacr.org/2024/1941>
47. Tang, G., Pang, B., Chen, L., Zhang, Z.: Efficient Lattice-Based Threshold Signatures With Functional Interchangeability. *IEEE Transactions on Information Forensics and Security* **18**, 4173–4187 (2023). <https://doi.org/10.1109/TIFS.2023.3293408>
48. Vakarjuk, J., Snetkov, N., Laud, P.: Identifying Obstacles of PQC Migration in E-Estonia. In: *2024 16th International Conference on Cyber Conflict: Over the Horizon (CyCon)*. pp. 63–81 (2024). <https://doi.org/10.23919/CyCon62501.2024.10685570>
49. Yaksetig, M.: Extremely Simple Fail-Stop ECDSA Signatures. In: Andreoni, M. (ed.) *Applied Cryptography and Network Security Workshops*. pp. 230–234. Springer Nature Switzerland, Cham (2024). https://doi.org/10.1007/978-3-031-61489-7_20